

Stoquify POS 9+ Execution Roadmap and Specialist-Skill Architecture Prompt

Reusable execution prompt • Stoquify POS transformation program • 20 July 2026

Date: 2026-07-20

Refined professional prompt

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Act as a senior enterprise POS architecture and delivery team:

- Preserve POS, payment, inventory, accounting, fiscal, receipt, offline-sync, close-assurance, and module-entitlement boundaries.
- Keep business and financial truth server-owned.
- Design workflow-first, role-aware cashier, supervisor, manager, accountant, owner, and support experiences only after authoritative state contracts exist.
- Enforce tenant isolation, RBAC, fresh authentication, maker-checker controls, auditability, redaction, safe errors, and cryptographic device identity.
- Keep statutory, currency, tax, fiscal, rounding, and OHADA/SYSCOHADA policy in expert-reviewed country packs rather than hard-coded UI logic.
- Treat test, reconciliation, operational, accessibility, hardware, and rollback evidence as release inputs—not documentation afterthoughts.

Mission

Using `docs/pos/STOQUIFY\_POS\_9\_PLUS\_EVALUATION\_AND\_ROADMAP\_2026-07-20.md` and its JSON scorecard, architect a complete, dependency-aware execution program that can move the Stoquify POS portal and workflow from the current 5.8/10 assessment to a defensible 9+/10 enterprise production standard.

Also design a reusable suite of specialized Codex skills for executing and certifying the program. The skill suite must divide work by trustworthy domain ownership, define orchestration and handoff contracts, reuse relevant installed skills where appropriate, and avoid a large overlapping “do everything” POS skill.

This run is planning and skill-architecture work only. Do not implement product changes or install the proposed skills.

Safety and governing principles

1. Financial truth takes precedence over visual polish.
2. No tender may be represented as paid without authoritative evidence.
3. No refund may be represented as completed without internal and external money-movement proof.
4. No offline event may disappear before an explainable terminal outcome.
5. No UI may claim scanner, network, provider, receipt, offline, or hardware readiness without objective detection and evidence.
6. Preserve the existing atomic sale, stock, cash, ledger, fiscal, audit, outbox, receipt-token, and shift-close strengths.
7. A high average score cannot compensate for failed money, session, tenant, authorization, device-trust, replay, reconciliation, or close gates.
8. Convert unresolved policies into explicit decision gates; do not silently assume them.
9. Preserve every unrelated working-tree change. Do not stage, commit, reset, clean, or modify product code.

Evidence-first discovery

1. Record the current Git status without altering it.
2. Read the complete POS evaluation, scorecard, relevant `what-next/` evidence, and current architecture graphs under `graphify-out/`.
3. Revalidate the authoritative boundaries under `app/`, `components/pos/`, `hooks/posHooks/`, `actions/pos/`, `services/pos/`, `services/payments/`, `lib/pos/`, `config/`, `prisma/`, and focused tests.
4. Treat reports and graphs as leads; current source wins when they disagree.
5. Label claims as Code-confirmed, Test-confirmed, Browser-observed, Report-derived, Externally dependent, Inference, Untested gap, or Product decision required.

Coordinated specialist workstreams

Use bounded read-only specialists for:

1. Payment, tender, refund, reconciliation, accounting, and fiscal truth.
2. Shift, drawer, terminal, session concurrency, and tenant boundaries.
3. Offline durability, device identity, replay, conflict recovery, and chaos behavior.
4. Cashier/supervisor UX, component decomposition, accessibility, responsive/handheld behavior, localization, receipts, and hardware ergonomics.
5. Security, RBAC, module entitlement, privacy, audit, and abuse cases.
6. Testing, observability, SLOs, field certification, rollout, rollback, and release assurance.
7. Skill architecture, orchestration, handoffs, reusable resources, and validation.

Each specialist must return evidence, gaps, work packages, dependencies, decisions, acceptance criteria, verification, migration/rollback risk, and proposed skill ownership. The coordinator must reconcile overlaps into one authoritative program.

Required roadmap phases

Phase 0 — decisions and scope freeze

Resolve or gate: STORE_CREDIT disposition; the physical one-active-session boundary; supported payment providers; provider acknowledgement/reversal semantics; offline tenders and risk limits; supported hardware/browser matrix; return/refund policy; country packs, currencies, minor units, rounding, tax, fiscal, and bilingual requirements; and measurable 9+ SLOs.

For every decision, provide options, trade-offs, recommendation, owner, deadline, and downstream blockers.

Phase 1 — capability honesty and containment

Plan reversible measures to disable unsupported store credit, remove fabricated readiness claims, hide unavailable receipt delivery, state the online-only checkout boundary honestly, and enforce consistent module entitlement and capability-aware UI.

Phase 2 — database and authorization invariants

Design the database-enforced active-session invariant, atomic terminal/drawer claims, tenant-owned device boundaries, entitlement/RBAC consistency, fresh authentication, maker-checker policy, safe migration, rollback, and concurrent negative tests.

Phase 3 — authoritative payment lifecycle

Define provider-aware initiated, pending, authorized, captured, declined, timed-out, reconciliation-pending, settled, reversed, refund-requested, refund-accepted, refund-settled, refund-failed, and manual-exception states. Map each state to provider evidence, idempotency, webhook security, UI, accounting, receipt, retry, reconciliation, close, and terminal-outcome behavior.

Phase 4 — offline durability and device trust

Design an IndexedDB transactional queue, durable per-event states, leases, stale-SYNCING recovery, per-event acknowledgement, terminal-outcome retention, multi-tab and restart safety, corruption/quota recovery, exact-once replay, registered device keys, canonical-payload signing, rotation/revocation, and forged/reordered/replayed event rejection.

Phase 5 — bounded offline selling

Only after Phase 4 gates pass, define tender/device/country/value/time eligibility, risk disclosure, durable-before-success capture, provisional fiscal/receipt treatment, reconnect/replay, close blockers, and global/tenant/store/device/tender kill switches.

Phase 6 — conflict and exception operations

Design a permissioned workbench for queue age, replay state, evidence, ownership, acknowledge, correct, retry, reject, quarantine, escalation, terminal resolution, immutable audit, and close-assurance linkage.

Phase 7 — returns, refunds, and voids

Design original-sale lookup, quantity eligibility, prior-refund deduction, partial/full/cross-shift/cross-location flows, original-tender disposition, cash availability, provider reversal, inventory return/write-off, fiscal corrections, journal/reconciliation effects, fresh auth, maker-checker, and immutable evidence.

Phase 8 — information architecture and decomposition

Define the target register status bar, scanner/search/catalog workspace, cart, customer/pricing controls, tender sidecar, exception layer, proof drawer, shift workspace, correction workbench, receipt governance, offline workbench, and hardware diagnostics. Provide routes, component boundaries, state ownership, service-owned read models, permission/error boundaries, and a low-risk migration from `ProfessionalPOSSystem`.

Phase 9 — cashier ergonomics and robust states

Plan scan-to-add, scanner buffering, debounced search, catalog paging/virtualization, keyboard-only checkout, touch targets, fast quantity edits, undo, controlled discounts, park/resume, notes, privacy-safe customer search, duplicate-charge protection, cart preservation, and truthful loading/empty/stale/denied/locked/offline/pending/declined/conflicted/retry states. Define measurable latency and recovery budgets.

Phase 10 — receipts, localization, and country packs

Plan real delivery adapters, availability detection, retry/evidence, provisional/final receipt status, reviewed English/French, country-pack money/tax/fiscal/rounding rules, zero-decimal currency correctness, cash rounding, fiscal corrections, and public-receipt redaction.

Phase 11 — accessibility, responsive, and hardware certification

Require WCAG 2.2 AA, axe, manual keyboard and screen-reader completion, focus/contrast/zoom/reflow/reduced-motion evidence, target viewport certification, device adapters, disconnect/reconnect and partial-failure recovery, and an evidence-backed supported-hardware matrix.

Phase 12 — reconciliation, observability, and operations

Design provider settlement/fees/refund reconciliation, stock/sale and drawer/shift tie-outs, fiscal monitoring, queue/conflict aging, payment and receipt metrics, SLOs, alerts, owners, runbooks, support bundles, incident response, rollback, and disaster-recovery rehearsal.

Phase 13 — differentiation after trust

Only after mandatory gates pass, plan handheld POS, line busting, trusted manager analytics, guided exception explanations, anomaly assistance, and country-specific payment/fiscal advantages. AI remains advisory, explainable, permissioned, and evidence-grounded.

Work-package contract

Create a stable work package for every P0-P3 proposal. Each must include: ID; title; priority; evidence; affected personas; problem and impact; architecture; exact likely boundaries; data/API/state/UI contracts; security/accounting/offline implications; dependencies; decision prerequisites; parallelization; effort; owner; migration; feature flag; rollback; abuse/failure cases; acceptance criteria; commands; evidence; definition of done; and release gate unlocked.

Include a traceability matrix proving that no evaluation recommendation was omitted.

Specialist-skill architecture contract

Propose the minimum coherent skill suite needed to execute the work packages. For every skill specify:

- Lowercase hyphenated name under 64 characters.
- Trigger-oriented description.
- Bounded responsibility and explicit non-responsibilities.
- Roadmap phases and work-package IDs owned.
- Required inputs and evidence.
- Step-by-step workflow.
- Expected artifacts and handoff contract.
- Mandatory pass/fail gates.
- Scripts, references, and assets worth bundling.
- Existing skills to reuse, extend, supersede, or keep separate.
- Dependencies and allowed parallel execution.
- Example trigger prompts.
- Validation and forward-test scenarios.

Define one thin orchestrator skill that sequences specialists, maintains a status register, prevents overlapping writes, enforces dependency and release gates, and never self-certifies specialist evidence.

Avoid skill proliferation. Merge responsibilities only when they share the same source of truth, lifecycle, verification method, and release authority. Split them when failure domains or approval authorities differ.

Program and release governance

Organize delivery into waves, milestones, dependency chains, parallel lanes, decision gates, hold points, release candidates, and rollback checkpoints. For each wave provide entry/exit criteria, work packages, relative duration, teams, verification, evidence, go/no-go authority, and rollback boundary. Do not invent calendar commitments without staffing and velocity evidence.

Retain these mandatory gates:

- G1 Money truth.
- G2 Session and drawer truth.
- G3 Offline durability.
- G4 Device trust.
- G5 Access, entitlement, and tenant isolation.
- G6 Accounting, inventory, fiscal, and reconciliation truth.
- G7 Operator safety, accessibility, responsive, and hardware readiness.
- G8 Observability, support, incident response, and rollback readiness.

Any failed mandatory gate produces NO-GO.

Required outputs

Save:

1. Execution roadmap: ``what-next/STOQUIFY_POS_9_PLUS_EXECUTION_ROADMAP_2026-07-20.md``
2. Machine-readable backlog: ``what-next/STOQUIFY_POS_9_PLUS_EXECUTION_BACKLOG_2026-07-20.json``
3. Proposed skill architecture:
``what-next/STOQUIFY_POS_9_PLUS_SPECIALIST_SKILL_ARCHITECTURE_2026-07-20.md``
4. Proposed skill catalog: ``what-next/STOQUIFY_POS_9_PLUS_SPECIALIST_SKILL_CATALOG_2026-07-20.json``

Do not create or install skill folders in this run. End with a recommended skill-creation sequence and the exact next prompt for building Wave 0 skills.

Verification

- Validate complete P0-P3 traceability.
- Validate JSON syntax and dependency references.
- Ensure every work package has acceptance, evidence, migration, rollback, and gate mapping.
- Ensure every proposed skill has a unique responsibility, trigger, handoff, and validation plan.
- Detect circular dependencies and orphan work packages.
- Confirm no product code, schema, configuration, tests, or application artifacts changed.
- Confirm nothing was staged or committed.

Success criteria

The run succeeds only when the roadmap provides a safe path from 5.8 to 9+, every proposal is owned by an implementation-ready work package, the specialist skills form a non-overlapping executable operating model, all eight gates have objective evidence requirements, unresolved policy is explicit, and product code remains untouched.

Non-goals

- Do not implement the roadmap.
- Do not install or modify Codex skills.
- Do not modify product code, schema, configuration, translations, or tests.
- Do not perform broad refactoring or unrelated cleanup.
- Do not stage, commit, push, or open a pull request.
- Do not invent provider, statutory, hardware, security, or production evidence.
- Do not certify the POS as 9+ from plans alone.

Finish with the recommended first execution wave, prerequisite decisions, parallel lanes, first mandatory hold point, exact evidence needed to advance, and skill-creation order.